

Agent SOD — Complete Guide

Agent SOD — Complete Guide: From Raw JSON to Violation Detection

Part 1: The Raw Data (What We Start With)

Where the data comes from

Microsoft Copilot Studio stores agent configuration in Dataverse. We fetch it via the Copilot365 backend API which produces two JSON files:

1. **agent-details.json** — one per agent (full config: components, topics, credentials, owner)
2. **all-agents-credentials.json** — cross-agent credential inventory (flat table of all credential bindings)

The Import API Payload

```
POST /api/v1/agent-sod/import
Content-Type: application/json
```

```
{
  "agents": [
    <contents of shadow-data-exfil-bot/03-agent-details.json>,
    <contents of accessreviewagent/03-agent-details.json>
  ],
  "credentials": <contents of _shared/all-agents-credentials.json>
}
```

What's inside agent-details.json (the fields we use)

```
{
  "id": "3fe5c1cd-8e2d-f111-88b4-7ced8d3b5df2",
  "name": "shadow-data-exfil-bot",
  "schemaName": "cr5c1_shadowdataexfilbot",
  "authMode": 2,
  "accessControlPolicy": 2,
  "generativeActionsEnabled": true,
  "agentOwner": {
    "fullName": "# AngadMehata",
    "upn": "AngadMehata@saviyntlivedev.onmicrosoft.com",
    "entraId": "ff1f1e1d-0b58-41d5-855c-1d0f808a8955"
  },
  "components": [
    {
      "id": "...",
      "type": 16,
      "typeName": "Knowledge Source",
      "name": "XYZ Sharepoint D0c",
      "urls": ["https://saviyntlivedev.sharepoint.com/Shared%20Documents"]
    },
    {
      "id": "...",
      "type": 16,
```

```

      "typeName": "Knowledge Source",
      "name": "https://pastebin.com/raw/something",
      "urls": ["https://pastebin.com/raw/something"]
    },
    {
      "id": "...",
      "type": 9,
      "typeName": "Topic",
      "name": "Agent-connected-via-topic",
      "schemaName": "cr5c1_shadowdataexfilbot.agent.Agent"
    }
  ],
  "topicDetails": [
    {
      "name": "Agent-connected-via-topic",
      "actions": [
        {
          "kind": "AgentDialog",
          "type": "connectedAgent",
          "agentName": "Agent-connected-via-topic"
        }
      ]
    }
  ]
}

```

Fields we extract: - `authMode` → determines AUTH_MODE_NONE flag (if value = 1) - `accessControlPolicy` → determines ACCESS_POLICY_MULTI_TENANT flag (if value = 3) - `generativeActionsEnabled` → determines GENERATIVE_ACTIONS flag - `agentOwner.entraId` → used for OWNER_COMPOSITE account linking - `components[]` where `type=16` → Knowledge Sources (kind determined from name/description) - `components[]` where `type=9` and `schemaName` contains `.agent.` → Chain edges - `topicDetails[].actions[]` where `type="connectedAgent"` → Chain edges (sub-agent invocations)

What's inside all-agents-credentials.json (the fields we use)

```

{
  "rows": [
    {
      "connectionId": "3135f073dfd24e33991ee84287c50528",
      "botId": "3fe5c1cd-8e2d-f111-88b4-7ced8d3b5df2",
      "botName": "shadow-data-exfil-bot",
      "connectorDisplayName": "mcplaygroundonline",
      "connectorEndpoint": "https://mcplaygroundonline.com",
      "isCustom": true,
      "isMcp": true,
      "authScheme": "none",
      "ownerAadObjectId": "ff1f1e1d-0b58-41d5-855c-1d0f808a8955"
    },
    {
      "connectionId": "4401c99c285c41ec9dc2932c1c92177e",
      "botId": "1605596a-8f34-f111-88b4-000d3a36bf2c",
      "botName": "AccessReviewAgent",
      "connectorDisplayName": "sportsdb",
      "connectorEndpoint": "https://api.sportdb.dev",
      "isCustom": true,
      "isMcp": false,
      "authScheme": "apiKey",
      "ownerAadObjectId": "ff1f1e1d-0b58-41d5-855c-1d0f808a8955"
    }
  ]
}

```

Fields we extract per credential: - `connectionId` → stored as attribute, used for CROSS_AGENT grouping - `botId` → filters which credentials belong to which agent - `connectorDisplayName` → CONNECTOR_NAME attribute - `connectorEndpoint` → CONNECTOR_ENDPOINT attribute - `authScheme` → AUTH_SCHEME attribute + HAS_ANONYMOUS_CRED flag if "none" - `isMcp` → IS_MCP attribute - `isCustom` → IS_CUSTOM attribute - `ownerAadObjectId` → OWNER_ENTRA_ID attribute

Part 2: How Data Gets Mapped to Tables

Step-by-step: What the import API does

When you call `POST /agent-sod/import`, here's exactly what happens for `shadow-data-exfil-bot`:

Step 1: Create the agent entitlement

```
INSERT INTO entitlement_values (ENTITLEMENT_VALUEKEY, ENTITLEMENTTYPEKEY, ENTITLEMENT_VALUE, ORPHAN)
VALUES (300000001, 600, 'shadow-data-exfil-bot', 0);
```

This is the agent itself — a node in the hierarchy graph.

Step 2: Create the star tcode (shared, one-time)

```
INSERT INTO entitlement_values (ENTITLEMENT_VALUEKEY, ENTITLEMENTTYPEKEY, ENTITLEMENT_VALUE, ORPHAN)
VALUES (300000000, 601, '*', 0);
```

Every agent gets linked to this. It's the "tcode gate" that SAP functions need (always passes).

Step 3: Link agent → star tcode

```
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY)
VALUES (300000001, 300000000);
```

Step 4: Parse knowledge sources from components[]

For each component with `type=16`:

```
-- KS: "XYZ Sharepoint D0c" (has URL containing "sharepoint" → kind = SharePoint)
INSERT INTO entitlement_values (ENTITLEMENT_VALUEKEY, ENTITLEMENTTYPEKEY, ENTITLEMENT_VALUE, ORPHAN)
VALUES (300200001, 601, 'shadow-data-exfil-bot::KS::XYZ Sharepoint D0c', 0);

-- Link agent → KS
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY)
VALUES (300000001, 300200001);

-- Attributes on the KS
INSERT INTO entitlement_objects (ENTITLEMENT_VALUEKEY, OBJECTKEY, FIELD_KEY, MINVALUE, MXVALUE, OBJECTDELETED)
VALUES (300200001, 9004, 1, 'SharePoint', 'SharePoint', 0); -- KS_KIND

INSERT INTO entitlement_objects (ENTITLEMENT_VALUEKEY, OBJECTKEY, FIELD_KEY, MINVALUE, MXVALUE, OBJECTDELETED)
VALUES (300200001, 9005, 1, 'https://saviyntlivedev.sharepoint.com/Shared%20Documents',
      'https://saviyntlivedev.sharepoint.com/Shared%20Documents', 0); -- KS_SITE

-- KS: "https://pastebin.com/raw/something" (public URL → kind = PublicSite)
INSERT INTO entitlement_values (ENTITLEMENT_VALUEKEY, ENTITLEMENTTYPEKEY, ENTITLEMENT_VALUE, ORPHAN)
VALUES (300200002, 601, 'shadow-data-exfil-bot::KS::https://pastebin.com/raw/something', 0);

INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY)
VALUES (300000001, 300200002);
```

```
INSERT INTO entitlement_objects (ENTITLEMENT_VALUEKEY, OBJECTKEY, FIELD_KEY, MINVALUE, MXVALUE, OBJECTDELETED)
VALUES (300200002, 9004, 1, 'PublicSite', 'PublicSite', 0);
```

```
INSERT INTO entitlement_objects (ENTITLEMENT_VALUEKEY, OBJECTKEY, FIELD_KEY, MINVALUE, MXVALUE, OBJECTDELETED)
VALUES (300200002, 9005, 1, 'pastebin.com', 'pastebin.com', 0);
```

Step 5: Parse credentials from all-agents-credentials.json (filtered by botId)

```
-- Credential: mcplaygroundonline (authScheme=none, isMcp=true)
INSERT INTO entitlement_values (ENTITLEMENT_VALUEKEY, ENTITLEMENTTYPEKEY, ENTITLEMENT_VALUE, ORPHAN)
VALUES (300300001, 601, 'shadow-data-exfil-bot::CRED:mcplaygroundonline', 0);

INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY)
VALUES (300000001, 300300001);

INSERT INTO entitlement_objects VALUES (300300001, 9003, 1, 'none', 'none', 0); -- AUTH_SCHEME
INSERT INTO entitlement_objects VALUES (300300001, 9001, 1, 'mcplaygroundonline', 'mcplaygroundonline', 0);
-- CONNECTOR_NAME
INSERT INTO entitlement_objects VALUES (300300001, 9006, 1, 'https://mcplaygroundonline.com',
'https://mcplaygroundonline.com', 0); -- ENDPOINT
INSERT INTO entitlement_objects VALUES (300300001, 9009, 1, '3135f073dfd24e33991ee84287c50528',
'3135f073dfd24e33991ee84287c50528', 0); -- CONNECTION_ID
INSERT INTO entitlement_objects VALUES (300300001, 9011, 1, 'true', 'true', 0); -- IS_MCP
INSERT INTO entitlement_objects VALUES (300300001, 9012, 1, 'true', 'true', 0); -- IS_CUSTOM
```

Step 6: Compute and link flags

Based on what we parsed: - Has SharePoint KS → HAS_INTERNAL_KS - Has PublicSite KS → HAS_EXTERNAL_KS - Has credential with authScheme="none" → HAS_ANONYMOUS_CRED - Has AgentDialog in topicDetails → HAS_CHAIN_EDGES - generativeActionsEnabled=true → GENERATIVE_ACTIONS

```
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY) VALUES (300000001, 300400001); --
HAS_CHAIN_EDGES
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY) VALUES (300000001, 300400003); --
GENERATIVE_ACTIONS
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY) VALUES (300000001, 300400007); --
HAS_ANONYMOUS_CRED
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY) VALUES (300000001, 300400010); --
HAS_EXTERNAL_KS
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY) VALUES (300000001, 300400011); --
HAS_INTERNAL_KS
```

Step 7: Create AGENT_ONLY user + account

```
-- Fake user for this agent
INSERT INTO users (USERKEY, USERNAME, ENABLED, PASSWORD) VALUES (9000001, 'AGENT::shadow-data-exfil-bot', 1,
'N/A');

-- Self-account (required by FK constraint: user_accounts.USERKEY must exist in accounts.ACCOUNTKEY)
INSERT INTO accounts (ACCOUNTKEY, NAME, ENDPOINTKEY, SYSTEMID, STATUS, ORPHAN)
VALUES (9000001, 'AGENT::shadow-data-exfil-bot', 300, 300, '1', 0);

-- Actual agent account
INSERT INTO accounts (ACCOUNTKEY, NAME, ENDPOINTKEY, SYSTEMID, STATUS, ORPHAN)
VALUES (3000001, 'AGENT::shadow-data-exfil-bot::acc', 300, 300, '1', 0);

-- Link user → account
INSERT INTO user_accounts (USERKEY, ACCOUNTKEY) VALUES (9000001, 3000001);

-- Assign agent entitlement to account
INSERT INTO account_entitlements1 (ACCOUNTKEY, ENTITLEMENT_VALUEKEY) VALUES (3000001, 300000001);
```

Step 8: Detect chain edges and create graph links

From `topicDetails[].actions[]` where `type="connectedAgent"` :

```
-- If AccessReviewAgent (300000002) is also imported:
INSERT INTO entitlements2 (ENTITLEMENT_VALUE1KEY, ENTITLEMENT_VALUE2KEY)
VALUES (300000001, 300000002); -- shadow-data-exfil-bot → AccessReviewAgent
```

Step 9: Detect cross-agent groups (shared connectionId)

If `connectionId=4401c99c...` appears for both shadow-data-exfil-bot AND AccessReviewAgent:

```
-- Virtual user for the group
INSERT INTO users (USERKEY, USERNAME, ENABLED, PASSWORD) VALUES (9000003, 'CROSS_AGENT::shared_cred_4401c99c',
1, 'N/A');
INSERT INTO accounts (ACCOUNTKEY, NAME, ENDPOINTKEY, SYSTEMID, STATUS, ORPHAN)
VALUES (9000003, 'CROSS_AGENT::shared_cred_4401c99c', 300, 300, '1', 0);

-- Group account
INSERT INTO accounts (ACCOUNTKEY, NAME, ENDPOINTKEY, SYSTEMID, STATUS, ORPHAN)
VALUES (3000003, 'CROSS_AGENT::shared_cred_4401c99c::acc', 300, 300, '1', 0);
INSERT INTO user_accounts (USERKEY, ACCOUNTKEY) VALUES (9000003, 3000003);

-- Assign BOTH agent entitlements to the group account
INSERT INTO account_entitlements1 (ACCOUNTKEY, ENTITLEMENT_VALUEKEY) VALUES (3000003, 300000001);
INSERT INTO account_entitlements1 (ACCOUNTKEY, ENTITLEMENT_VALUEKEY) VALUES (3000003, 300000002);
```

Part 3: How the Engine Evaluates (What Happens on POST /evaluate)

The engine's 5-phase pipeline runs UNMODIFIED:

```
Phase 0: Load config
- Loads risks from rulesetKey=203
- Loads functions (SAP type and NONSAP type)
- Loads function_objects (SAP conditions)
- Loads function_entitlements (NonSAP conditions)

Phase 1: Load graph + resolve access
- Loads entitlements2 WHERE securitySystemKey=300
- Loads account_entitlements1 for accounts in system 300
- For each account: BFS through the graph to build resolvedEntitlements[]
- Builds auth index from entitlement_objects on resolved entitlements

Phase 2: Evaluate functions (parallel)
- For each function, for each user: does this user satisfy this function?
- SAP functions: check auth index for matching objectKey/fieldKey/value
- NonSAP functions: binarySearch for entitlement key in resolved set
- Result: one BitSet per function (bit i = user i satisfies it)

Phase 3: Detect violations
- For each risk: AND its function BitSets
- Users with bit set in ALL function BitSets = violators

Phase 4: Write violations to sodrisks_new_job
```

Concrete example: shadow-data-exfil-bot evaluated**Phase 1: BFS resolution**

Starting from account 3000001 → assigned entitlement 300000001 (shadow-data-exfil-bot).

BFS walks the graph:

```
300000001 (agent)
  → 300000000 (star tcode *)
  → 300200001 (KS: SharePoint)
  → 300200002 (KS: pastebin.com)
  → 300200003 (KS: evil-external-site.com)
  → 300200004 (KS: Bing Custom Search)
  → 300300001 (CRED: mcpplayground, authScheme=none)
  → 300300002 (CRED: sportsdb)
  → 300300003 (CRED: Microsoft Copilot Studio)
  → 300400001 (FLAG: HAS_CHAIN_EDGES)
  → 300400003 (FLAG: GENERATIVE_ACTIONS)
  → 300400007 (FLAG: HAS_ANONYMOUS_CRED)
  → 300400010 (FLAG: HAS_EXTERNAL_KS)
  → 300400011 (FLAG: HAS_INTERNAL_KS)
  → 300000002 (sub-agent: AccessReviewAgent) ← CHAIN EDGE
    → 300000000 (star tcode - already visited, skip)
    → 300200005 (KS: trust.saviynt.com)
    → 300200006 (KS: docs.saviyntcloud.com)
    → 300300004 (CRED: Dynamics 365)
    → 300300005 (CRED: ServiceNow)
    → ...
```

resolvedEntitlements for user 9000001:

```
[300000000, 300000001, 300000002, 300200001, 300200002, 300200003, 300200004,
300200005, 300200006, 300300001, 300300002, 300300003, 300300004, 300300005,
300400001, 300400003, 300400007, 300400010, 300400011, ...]
```

Auth index built from entitlement_objects on all resolved keys:

```
compositeKey(9003, 1) → [AuthEntry(300300001, 9003, 1, "none", "none")]
compositeKey(9004, 1) → [AuthEntry(300200001, 9004, 1, "SharePoint", "SharePoint"),
                        AuthEntry(300200002, 9004, 1, "PublicSite", "PublicSite"),
                        AuthEntry(300200003, 9004, 1, "PublicSite", "PublicSite")]
compositeKey(9001, 1) → [AuthEntry(300300001, 9001, 1, "mcpplayground", "mcpplayground"),
                        AuthEntry(300300002, 9001, 1, "sportsdb", "sportsdb")]
...
```

Phase 2: Function evaluation**NonSAP Function “Can Read Internal Data” (functionKey=652):**

```
Condition: HasEntitlement(300400011)
Check: binarySearch(resolvedEntitlements, 300400011)
Result: FOUND → function SATISFIED ✓
```

NonSAP Function “Can Send Externally” (functionKey=653):

```
Condition: HasEntitlement(300400010) OR HasEntitlement(300400007)
Check left: binarySearch(resolvedEntitlements, 300400010) → FOUND
Result: short-circuit OR → function SATISFIED ✓
```

SAP Function “Has No-Auth Connector” (functionKey=658):

Condition: tcodeKey=300000000, objectKey=9003, fieldKey=1, min="none", max="none"

Step 1 – Tcode gate: Is 300000000 in resolvedEntitlements?
binarySearch(resolvedEntitlements, 300000000) → FOUND ✔️ (star tcode always passes)

Step 2 – Auth check: Does auth index have (9003, 1) with value overlapping ["none","none"]?
compositeKey = 9003 * 100000 + 1 = 900300001
Look up auth index → finds AuthEntry(300300001, 9003, 1, "none", "none")
ValueMatcher.rangesOverlap("none", "none", "none", "none") → TRUE ✔️

Result: function SATISFIED ✔️

SAP Function “Has SharePoint KS” (functionKey=659):

Condition: tcodeKey=300000000, objectKey=9004, fieldKey=1, min="SharePoint", max="SharePoint"

Step 1 – Tcode gate: 300000000 in resolved? → YES ✔️

Step 2 – Auth check: (9004, 1) with value overlapping ["SharePoint","SharePoint"]?
Finds AuthEntry(300200001, 9004, 1, "SharePoint", "SharePoint")
rangesOverlap("SharePoint", "SharePoint", "SharePoint", "SharePoint") → TRUE ✔️

Result: function SATISFIED ✔️

Phase 3: Violation detection

Risk “Data Exfiltration Path” (riskId=777): F652 AND F653

F652 BitSet: [1, 0, 1] (user 0=shadow satisfied, user 1=AccessReview not, user 2=cross-agent satisfied)
F653 BitSet: [1, 0, 1]
AND result: [1, 0, 1] → users 0 and 2 VIOLATE

Risk “No-Auth Connector + Internal Data” (riskId=780): F658 AND F659

F658 BitSet: [1, 0, 1] (shadow has none-auth cred)
F659 BitSet: [1, 0, 1] (shadow has SharePoint KS)
AND result: [1, 0, 1] → users 0 and 2 VIOLATE

Phase 4: Write violations

```
INSERT INTO sodrisks_new_job (USERIDENTIFIER, RISKKEY, ENDPOINTKEY, STATUS, ...)
VALUES (9000001, 777, 0, 1, ...); -- shadow-data-exfil-bot violates Data Exfiltration
VALUES (9000003, 777, 0, 1, ...); -- cross-agent group violates Data Exfiltration
VALUES (9000001, 780, 0, 1, ...); -- shadow-data-exfil-bot violates No-Auth+Internal
...
```

Part 4: When to Use SAP-style vs NonSAP-style Functions

Decision rule:

Question	Use SAP	Use NonSAP
“Does the agent have a tool/KS/cred with attribute X = value Y?”	✔️	

Question	Use SAP	Use NonSAP
"Does the agent have a tool with attribute X = A AND attribute Y = B?"	✓	
"Does the agent have property Z?" (boolean yes/no)		✓
"Does the agent have property A OR property B?"		✓

SAP-style: For attribute matching on tools/KS/credentials

When: You want to check if ANY entitlement in the resolved set has specific attribute values.

How it works: The engine builds an auth index from ALL `entitlement_objects` rows on resolved entitlements. Then checks if the index contains entries matching your `objectKey/fieldKey/value` conditions.

Examples:

```
"Has a connector named Dynamics 365"
→ objectKey=9001, fieldKey=1, min="Dynamics 365", max="Dynamics 365"

"Has a credential with auth scheme = none"
→ objectKey=9003, fieldKey=1, min="none", max="none"

"Has a knowledge source of kind SharePoint"
→ objectKey=9004, fieldKey=1, min="SharePoint", max="SharePoint"

"Has a connector starting with 'Dynamics'" (prefix match)
→ objectKey=9001, fieldKey=1, min="Dynamics", max="Dynamics~"

"Has Maker credential on Dynamics" (AND – same group)
→ group 1: objectKey=9002, fieldKey=1, min="Maker", max="Maker"
→ group 1: objectKey=9001, fieldKey=1, min="Dynamics", max="Dynamics~"

"Has SharePoint KS OR Dataverse KS" (OR – different groups)
→ group 1: objectKey=9004, fieldKey=1, min="SharePoint", max="SharePoint"
→ group 2: objectKey=9004, fieldKey=1, min="Dataverse", max="Dataverse"
```

The star tcode trick: Every SAP condition uses `tcodeKey=300000000`. This is the star tcode that's always in every agent's resolved set. It makes the tcode-ownership gate always pass, so the evaluation is purely attribute-based.

NonSAP-style: For boolean property checks

When: You want to check if the agent HAS a specific flag (computed at import time).

How it works: The engine does `binarySearch(resolvedEntitlements, flagKey)`. If found → satisfied.

Examples:

```
"Agent has chain edges"
→ HasEntitlement(300400001)

"Agent has generative actions enabled"
→ HasEntitlement(300400003)

"Agent has anonymous credential"
→ HasEntitlement(300400007)

"Agent has external KS OR anonymous cred" (OR)
→ position 1: HasEntitlement(300400010), nextOperator="||"
```



```
→ position 2: HasEntitlement(300400007)
```

```
"Agent has chain edges AND generative actions" (AND)
```

```
→ position 1: HasEntitlement(300400001), nextOperator="&&"
```

```
→ position 2: HasEntitlement(300400003)
```

Mixed risks: SAP function + NonSAP function in same risk

A risk can pair ANY combination:

```
Risk "No-Auth Connector + Chain Delegation":
```

```
function1 = SAP function checking AUTH_SCHEME = "none"
```

```
function2 = NonSAP function checking HAS_CHAIN_EDGES flag
```

The engine evaluates each function via its own path (SAP or NonSAP), then ANDs the resulting BitSets. Works perfectly.

Part 5: All Five Risk Types — How Each Works

AGENT_ONLY

Question: “Does this single agent have a dangerous combo?”

Setup: Agent gets its own fake user + account. Account assigned to agent entitlement.

BFS resolves: Only this agent's tools/KS/creds/flags.

Example violation: shadow-data-exfil-bot has BOTH internal KS (SharePoint) AND external path (pastebin + anonymous cred) → Data Exfiltration.

```
User: AGENT::shadow-data-exfil-bot (userKey=9000001)
```

```
Account: 3000001 → assigned to entitlement 300000001
```

```
BFS: resolves shadow-data-exfil-bot's own capabilities only
```

```
Functions: F1(internal KS) ✅ + F2(external path) ✅ → VIOLATION
```

CHAIN

Question: “Does parent + sub-agent together have a dangerous combo?”

Setup: Edge in entitlements2: parent_agent_ent → sub_agent_ent

BFS resolves: Parent's tools + sub-agent's tools (BFS traverses the edge naturally).

Example: shadow-data-exfil-bot invokes AccessReviewAgent. BFS from shadow resolves BOTH agents' tools. If the combined set triggers a risk that neither alone would → chain violation.

```
entitlements2: 300000001 → 300000002 (shadow → AccessReview)
```

```
User: AGENT::shadow-data-exfil-bot (userKey=9000001)
```

```
BFS from 300000001:
```

```
→ shadow's own tools/KS/creds
```

```
→ 300000002 (AccessReviewAgent) → AccessReview's tools/KS/creds
```

```
resolvedEntitlements = UNION of both agents' capabilities
```

No special chain pass needed. The graph handles it.

CROSS_AGENT

Question: “Do multiple agents sharing a resource together form a conflict?”

Setup: Virtual user + account assigned to MULTIPLE agent entitlements.

BFS resolves: Union of all grouped agents' tools.

How groups are detected: During import, credentials with the same `connectionId` across different agents are grouped. A virtual account is created assigned to all agents in the group.

```
connectionId "4401c99c..." appears on:
- shadow-data-exfil-bot (botId=3fe5c1cd...)
- AccessReviewAgent (botId=1605596a...)

Virtual user: CROSS_AGENT::shared_cred_4401c99c (userKey=9000003)
Virtual account: 3000003
  → assigned to 300000001 (shadow-data-exfil-bot)
  → assigned to 300000002 (AccessReviewAgent)

BFS resolves BOTH agents' full capability trees into one set.
```

OWNER_COMPOSITE

Question: “Does the owner’s personal access + agent’s tools together form a conflict?”

Setup: The owner (who already exists in ECM with SAP accounts) gets an ADDITIONAL account in system 300 assigned to the agent entitlement.

BFS resolves: Owner’s SAP roles/tcodes (from system 5 account) + agent’s tools/KS/creds (from system 300 account) → merged into ONE resolved set.

Example: Manish owns AccessReviewAgent. Manish has SAP AP Approver role. AccessReviewAgent has Maker credential on Dynamics 365. Neither alone is a violation. Together = owner can approve payments AND agent can initiate them on his behalf.

```
Manish (userKey=77) already has:
  Account in system 5: SAP roles → tcodes → auth objects

We ADD:
  Account in system 300: assigned to 300000002 (AccessReviewAgent)

Engine loads BOTH accounts for userKey=77.
resolvedEntitlements = SAP stuff + agent stuff (merged)

Risk: "Owner has AP Approver" (SAP function on system 5 data)
      + "Agent has Maker on Finance" (SAP function on system 300 data)
      → FIRES because both are in the same resolved set for user 77
```

Currently: Not firing in test because sample owners don’t exist in this ECM DB. Works when real owner users are present.

INVOKER_COMPOSITE

Question: “Does the invoking user’s access + agent’s OBO tools form a conflict?”

Setup: Exactly same as OWNER_COMPOSITE. Each invoker gets an account in system 300 assigned to the agent entitlement.

Example: Kunal can chat with AccessReviewAgent (he’s in directShares). Kunal has SAP Vendor Master role. Agent has tools that can create purchase orders. Together = invoker can create vendors AND agent creates POs against those vendors.

```
Kunal (userKey=88) already has:
  Account in system 5: SAP Vendor Master role
```

We ADD:

Account in system 300: assigned to 300000002 (AccessReviewAgent)

Engine merges Kunal's SAP ents + agent tools.

Risk fires if the combined set satisfies both functions.

Currently: Needs invoker group expansion (Graph API to get group members) to populate.

Part 6: Complete API Examples (Copy-Paste Ready)

Full workflow from scratch

```
BASE="http://localhost:9220/sod-eval/api/v1/agent-sod"

# — STEP 1: Import agent data —————
# Build payload from handoff sample files
python3 -c "
import json
with open('/Users/arin.mallanna/Downloads/handoff/samples/shadow-data-exfil-bot/03-agent-details.json') as f:
    a1 = json.load(f)
with open('/Users/arin.mallanna/Downloads/handoff/samples/accessreviewagent/03-agent-details.json') as f: a2 =
    json.load(f)
with open('/Users/arin.mallanna/Downloads/handoff/samples/_shared/all-agents-credentials.json') as f: creds =
    json.load(f)
print(json.dumps({'agents': [a1, a2], 'credentials': creds}))
" > /tmp/agent-payload.json

curl -X POST "$BASE/import" -H "Content-Type: application/json" -d @/tmp/agent-payload.json
# {"agentsImported":2,"crossAgentGroups":1}

# — STEP 2: Create ruleset —————
curl -X POST "$BASE/rulesets" -H "Content-Type: application/json" \
  -d '{"name":"Agent SOD Rules","description":"Agent SOD"}'
# {"rulesetKey":203}

# — STEP 3: Create functions —————

# NonSAP: Can Read Internal Data (has internal KS flag)
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Can Read Internal Data","type":"NONSAP","rulesetKey":203}'
# {"functionKey":652}
curl -X POST "$BASE/functions/652/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400011,"position":1}'

# NonSAP: Can Send Externally (has external KS OR anonymous cred)
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Can Send Externally","type":"NONSAP","rulesetKey":203}'
# {"functionKey":653}
curl -X POST "$BASE/functions/653/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400010,"position":1,"nextOperator":"||"}'
curl -X POST "$BASE/functions/653/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400007,"position":2}'

# NonSAP: Has Chain Edges
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Has Chain Edges","type":"NONSAP","rulesetKey":203}'
# {"functionKey":654}
curl -X POST "$BASE/functions/654/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400001,"position":1}'
```

```

# NonSAP: Has Generative Actions
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Has Generative Actions","type":"NONSAP","rulesetKey":203}'
# {"functionKey":656}
curl -X POST "$BASE/functions/656/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400003,"position":1}'

# SAP: Has No-Auth Connector (attribute match on AUTH_SCHEME)
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Has No-Auth Connector","type":"SAP","rulesetKey":203}'
# {"functionKey":658}
curl -X POST "$BASE/functions/658/conditions" -H "Content-Type: application/json" \
  -d '{"groupKey":1,"tcodeKey":300000000,"objectKey":9003,"fieldKey":1,"minValue":"none","maxValue":"none"}'

# SAP: Has SharePoint KS (attribute match on KS_KIND)
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Has SharePoint KS","type":"SAP","rulesetKey":203}'
# {"functionKey":659}
curl -X POST "$BASE/functions/659/conditions" -H "Content-Type: application/json" \
  -d '{"groupKey":1,"tcodeKey":300000000,"objectKey":9004,"fieldKey":1,"minValue":"SharePoint","maxValue":"SharePoint"}'

# SAP: Has Maker on Dynamics (AND – two conditions same group)
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Maker on Dynamics","type":"SAP","rulesetKey":203}'
# {"functionKey":660}
curl -X POST "$BASE/functions/660/conditions" -H "Content-Type: application/json" \
  -d '{"groupKey":1,"tcodeKey":300000000,"objectKey":9002,"fieldKey":1,"minValue":"Maker","maxValue":"Maker"}'
curl -X POST "$BASE/functions/660/conditions" -H "Content-Type: application/json" \
  -d '{"groupKey":1,"tcodeKey":300000000,"objectKey":9001,"fieldKey":1,"minValue":"Dynamics","maxValue":"Dynamics~"}'

# — STEP 4: Create risks —————

# Data Exfiltration (NonSAP + NonSAP)
curl -X POST "$BASE/risks" -H "Content-Type: application/json" \
  -d '{"name":"Data Exfiltration Path","rulesetKey":203,"functionKeys":[652,653]}'

# Chain + Anonymous Cred (NonSAP + NonSAP)
curl -X POST "$BASE/risks" -H "Content-Type: application/json" \
  -d '{"name":"Chain with Anonymous Cred","rulesetKey":203,"functionKeys":[654,653]}'

# No-Auth + Internal Data (SAP + SAP)
curl -X POST "$BASE/risks" -H "Content-Type: application/json" \
  -d '{"name":"No-Auth + Internal Data","rulesetKey":203,"functionKeys":[658,659]}'

# Internal Data + Chain (Mixed SAP + NonSAP)
curl -X POST "$BASE/risks" -H "Content-Type: application/json" \
  -d '{"name":"Internal Data + Chain","rulesetKey":203,"functionKeys":[659,654]}'

# Generative + No-Auth (Mixed NonSAP + SAP)
curl -X POST "$BASE/risks" -H "Content-Type: application/json" \
  -d '{"name":"Autonomous + No-Auth","rulesetKey":203,"functionKeys":[656,658]}'

# — STEP 5: Evaluate —————
curl -X POST "$BASE/evaluate" -H "Content-Type: application/json" -d '{"rulesetKey":203}'

# — STEP 6: Check violations —————
curl -s "$BASE/violations?rulesetKey=203" | python3 -m json.tool

```

Part 7: Advanced Patterns

Pattern: Prefix matching (connector names starting with X)

```
# "Any connector starting with Saviynt"
{"groupKey":1,"tcodeKey":300000000,"objectKey":9001,"fieldKey":1,"minValue":"Saviynt","maxValue":"Saviynt~"}
# Matches: "Saviynt Gateway", "Saviynt Acess Gateway", "Saviynt Gateway - Salesforce01"
```

Pattern: Multiple OR alternatives (SAP-style)

```
# "KS is SharePoint OR Dataverse OR BingCustomSearch" (3 groups = OR)
{"groupKey":1,"tcodeKey":300000000,"objectKey":9004,"fieldKey":1,"minValue":"SharePoint","maxValue":"SharePoint"}
{"groupKey":2,"tcodeKey":300000000,"objectKey":9004,"fieldKey":1,"minValue":"Dataverse","maxValue":"Dataverse"}
{"groupKey":3,"tcodeKey":300000000,"objectKey":9004,"fieldKey":1,"minValue":"BingCustomSearch","maxValue":"BingCustomSea"}
```

Pattern: AND across different attributes (same group)

```
# "Has MCP connector that is custom AND anonymous"
{"groupKey":1,"tcodeKey":300000000,"objectKey":9011,"fieldKey":1,"minValue":"true","maxValue":"true"}
{"groupKey":1,"tcodeKey":300000000,"objectKey":9003,"fieldKey":1,"minValue":"none","maxValue":"none"}
# Both must match on SOME entitlement in the resolved set's auth index
```

Pattern: 3-function risk

```
# "Can read internal + can send external + has chain edges" (all three must be true)
curl -X POST "$BASE/risks" -H "Content-Type: application/json" \
  -d '{"name":"Full Exfil Chain","rulesetKey":203,"functionKeys":[652,653,654]}'
```

Pattern: NonSAP AND condition

```
# "Has chain edges AND generative actions" (both flags must be present)
curl -X POST "$BASE/functions" -H "Content-Type: application/json" \
  -d '{"name":"Chain + Generative","type":"NONSAP","rulesetKey":203}'
# functionKey: 661
curl -X POST "$BASE/functions/661/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400001,"position":1,"nextOperator":"&&"}'
curl -X POST "$BASE/functions/661/conditions" -H "Content-Type: application/json" \
  -d '{"entitlementKey":300400003,"position":2}'
```

Pattern: Testing OWNER_COMPOSITE (with synthetic owner data)

To test owner composite without real ECM data, create a synthetic owner:

```
-- Create a test user that represents an agent owner
INSERT INTO users (USERKEY, USERNAME, ENABLED, PASSWORD) VALUES (77, 'manish.acharya', 1, 'N/A');
INSERT INTO accounts (ACCOUNTKEY, NAME, ENDPOINTKEY, SYSTEMID, STATUS, ORPHAN) VALUES (77, 'manish.acharya',
300, 300, '1', 0);

-- Give them a "SAP AP Approver" entitlement in system 5
INSERT INTO accounts (ACCOUNTKEY, NAME, ENDPOINTKEY, SYSTEMID, STATUS, ORPHAN) VALUES (5077,
'manish.acharya@sap', 7, 5, '1', 0);
INSERT INTO user_accounts (USERKEY, ACCOUNTKEY) VALUES (77, 5077);
INSERT INTO account_entitlements1 (ACCOUNTKEY, ENTITLEMENT_VALUEKEY) VALUES (5077, <SAP_AP_APPROVER_KEY>);

-- Now re-import agents (the import will find userKey=77 for owner "manish.acharya")
```

```
-- and create an agent account linked to userKey=77
-- Engine will merge SAP ents + agent tools for user 77
```

Part 8: What Gets Stored Where (Table Summary)

Table	What we put in it	Purpose
securitysystems	Row with SYSTEMKEY=300	Isolates agent data
endpoints	Row with ENDPOINTKEY=300, linked to system 300	Endpoint for agent types
entitlement_types	Two rows: Agent (600), AgentCapability (601)	Type classification
entitlement_values	One row per agent, tool, KS, credential, flag	The nodes in the graph
entitlements2	Edges: agent→tools, agent→KS, agent→creds, agent→flags, agent→sub-agent	The hierarchy graph
entitlement_objects	Attributes on tools/KS/creds (connector name, auth scheme, KS kind, etc.)	What SAP functions match against
users	Fake users for agents + cross-agent groups	Evaluation subjects
accounts	Self-accounts (FK requirement) + agent accounts	Links users to entitlements
user_accounts	Links users to their agent accounts	Engine uses this to find user's accounts
account_entitlements1	Links accounts to agent entitlements	Starting point for BFS
functions	Function definitions (SAP or NONSAP type)	What capabilities to detect
function_objects	SAP conditions (objectKey, fieldKey, value ranges)	Attribute matching rules
function_entitlements	NonSAP conditions (entitlement key presence)	Flag presence rules
risks	Conflict definitions (function1 + function2 + ...)	What combinations are violations
rulesets	Rule collection	Groups risks together
sodrisks_new_job	OUTPUT: Detected violations	What the engine writes
sodrisk_entitlement_new_job	OUTPUT: Violation evidence detail rows	Per-tcode evidence

Part 9: Key Insight — Why This Works Without Engine Changes

The SOD evaluation engine was built to answer: “Does this user have a dangerous combination of access?”

It doesn't care WHAT the entitlements represent. It just: 1. Walks a graph (BFS) 2. Collects attributes (auth index) 3. Checks conditions (SAP: attribute match, NonSAP: presence check) 4. ANDs BitSets (violation detection)

By mapping agents into the same data structures: - Agent = entitlement (a node in the graph) - Tool/KS/Cred = child entitlement (children in the graph) - Tool attributes = entitlement_objects (same format as SAP auth objects) - Boolean flags = leaf entitlements (same as NonSAP entitlement presence) - Agent account = evaluation subject (same as a human user's account)

The engine processes it identically to how it processes SAP roles → tcodes → auth objects. It doesn't know or care that the data represents AI agents instead of human users.